PHP Programming

UNIT-I

Introduction to PHP: Evaluation of PHP, Basic Syntax, Defining variable and constant, PHP Data type, Operator and Expression, Decisions and loop: Making Decisions, Doing Repetitive task with looping, Mixing Decisions and looping with Html, Function: What is a function, Define a function, Call by value and Call by reference, Recursive function, String Creating and accessing, String Searching & Replacing String, Formatting String, String Related Library function

Internet: The term "Internet" refers to a group of linked networks. or A way to link a computer to any other computer at any location all over the world with specialized servers and routers. When there are two PCs linked via the Internet, they are able to send and receive a variety of data including text, images, audio, video, and computer programs. TCP/IP is used by the Internet to send data via a variety of media.

World Wide Web (WWW): The WWW is a global network that internet material that can be seen over HTTP and is formatted in HTML. The phrase describes all of the accessible, interconnected HTML pages using the Internet. In 1991, the World Wide Web was first conceptualized by Tim Berners-Lee.

Website: A website serves as a hub for different web pages that are all connected and available simply going to the official website of the webpage with a web browser.

Webpage: A web page, often known as a website, is a document that is typically written in HTML (Hypertext Markup Language) that is viewable using an Internet browser via the Internet or other networks. A web page can have text, pictures, and linkages to other web sites and files. It can be viewed by entering its URL address.

Web pages are of two types:

1. Static Web Pages – Does not contain logic
2. 2. Dynamic Web Pages - Contains logic

Web server: A web server is a computer that provides Web pages for delivery. Each domain name and IP address belong to a Web server. For instance, when you type in the URL http://www.webopedia.com/index.html in your browser, a request is sent to the web server with the webopedia.com domain name. The page index.html is then fetched by the server and sent to your browser. By installing server software and connecting the device to the Internet, any computer may be made into a Web server.

PHP:

- PHP is an acronym for "PHP: Hypertext Preprocessor."
- PHP is a scripting language that is commonly used and open source.
- PHP scripts are executed on the server.
- PHP is free to download and use.

In 1994, Rasmus Lerdorf implemented PHP, a server-side scripting language, by utilizing the PERL and C languages. In order to monitor the total number of visitors on his server, he implemented PHP 1.0.PHP is an acronym for Personal Home Page. It is also known by the alias Hypertext Preprocessor.

PHP files are characterized by the presence of text, HTML, CSS, JavaScript, and PHP code. The server executes PHP code, which is then returned to the browser in the form of ordinary HTML. PHP files are identified by the ".php" extension.

PHP Capabilities:

- PHP has the ability to generate dynamic page content.
- PHP can collect form data.
- PHP can transmit and receive cookies.
- PHP has the ability to add, delete, and modify data in your database.
- PHP can create, open, read, write, delete, and close files on the server.
- PHP can be employed to regulate user access. Additionally, it has the capability to encrypt data.

Features of PHP:

PHP is a server-side open-source scripting language that can be freely downloaded and used. The primary reason for its widespread popularity is its simplicity and open source nature. It is the most frequently used scripting language on a global scale.

Simple: It is extensively used worldwide due to its ease of use and simplicity in comparison to other scripting languages. PHP offers a plethora of pre-defined functions to safeguard your data. It is also compatible with a multitude of third-party applications, and PHP can be readily integrated with others.In PHP scripts, there is no requirement to include libraries such as C or special compilation directives like Java.

Cross-Web Server: It is compatible with a variety of web servers, including Apache, Tomcat, and IIS.

Case Sensitive: PHP is a scripting language that is case-sensitive at the time of variable declaration. In PHP, the case of all keywords (e.g. if, else, while, echo, etc.), classes, functions, and user-defined functions is not a factor.

Open Source: Open source software is available for free installation and use, eliminating the need to pay for PHP.

Flexibility: PHP is a highly adaptable language due to its embedded nature, which allows for the embedding of PHP scripts with a variety of formats, including HTML, JavaScript, WML, and XML. Your PHP script can be executed on any device, including mobile phones, tablets, laptops, and PCs, thanks to the fact that it is executed on the server and then sent to the browser of the device.

Error Reporting: PHP has several predefined error reporting constants that are used to produce a warning or error notice. PHP is a loosely typed language that allows for the use of variables without the need to declare their data type. Given the nature of the data and its value, it will be retrieved at the time of execution.

Protocol Independent: PHP supports a significant number of key protocols, including POP3, IMAP, and LDAP.PHP4 introduced support for Java and distributed object architectures (COM and CORBA), thereby enabling the first instance of n-tier development.

Faster: It is more efficient than other scripting languages, such as ASP and JSP.

Interpreted: It is an interpreted language, which means that compilation is unnecessary.

Platform-independent: PHP code will be executed on all platforms, including Linux, Unix, Mac OS X, and Windows.

Cross-Database Integration: It is compatible with numerous widely used databases, such as MySQL, Oracle, Sybase, Informix, and Microsoft SQL Server.

Security: It facilitates the implementation of various security functionalities, including authentication, one-way encryption, and two-way encryption, to ensure the security of applications.

Experience: If you have a background in programming, you will have no difficulty comprehending the PHP syntax. Additionally, the majority of PHP syntax is derived from other languages, such as Pascal or C, which allows for the creation of PHP scripts.

Real-Time Access Monitoring: PHP generates a summary of the user's most recent accesses to facilitate access monitoring.

PHP Versions

PHP 1.0 (1994-95) - Not a server-side script - Referred to as "Personal Home Page"

PHP 2.0 was released in 1997. - Script that is partially server-side Cross-database

PHP 3.0 (1998) A script that is entirely server-side Hypertext Preprocessor is a cross-platform tool.

PHP 4.0 (2000) - - Cross-platform web server Zend Engine 1.0, which is the runtime engine of PHP, is introduced.

PHP 5.0 (2004-05) Supports the concepts of object-oriented programming Zend 2.0 is introduced.

PHP 6.0 (2005) - Implemented Unicode support

PHP 7.0 (2015) - The level of support for XML and Web services has been enhanced.

Zend 3.0 is introduced. Decreased memory utilization

PHP'S MOST COMMON USES

- It is employed to develop dynamic websites.
- It is capable of processing forms, with the ability to collect data from files and save it to a file.
- PHP can be employed to establish a designated area of your website for members.
- A registration page can be generated for a user using PHP.
    - To interact with a web server (e.g., Apache) andTo interact with a back-end or database server, such as MySQL
    - To establish the business logical layers (one or more)
    - Set cookies and access the Cookies variable
    - PHP allows users to restrict access to specified web pages.
    - PHP is typically employed to generate HTML code for the browser. • It is also employed to transmit and receive emails.
    - PHP can be employed to determine the current date and subsequently generate a monthly

calendar.

- website's visitors can be counted using PHP.
- Elements within your database can be added, deleted, or modified using PHP.
- By employing PHP, it is possible to restrict users' access to specific pages of your website.
- Data can be encrypted.
- PHP is capable of performing system functions, such as the creation, opening, reading, writing, and closing of files on a system.

PHP Files

- PHP file have a .php or .php3 extension; you also have the option of changing the default file extension using the web server's configuration file.
- A .php file should have all the PHP code within <?php (opening tag) and ?> (closing tag), then It will be considered as PHP language code and parsed accordingly. It is also useful when you want to write PHP code with HTML.
- PHP can be embedded inside HTML but in a slightly different fashion. In between the HTML tags where ever you want to make use of PHP code, you will have to use <?php /*PHP code*/ ?> and PHP will parse this.
- All PHP statements must end with a ; (semicolon) which is also called statement terminator. It is essential because semicolon tells the PHP interpreter to read this line as a single statement, and without this, you will see error messages while interpreting the code.

The php.ini File

php.ini is a plain text file that configures PHP settings. PHP interpreter reads the php.ini file to determine what settings to use. We will refer to this file from time to time in the course, but now, it is enough that you are aware of its existence.

How PHP Works

- When a user navigates in her browser to a page that ends with a .php extension, the browser sends an HTTP request to the web server, for example, you are typing the URL of the file index.php and hit enter, the browser will send the request to server and server will start finding this file on its file

system. If the server finds the file, the server will send this file to PHP interpreter. Otherwise, the server will generate ERROR 404, which is also known as file not found.

- The web server only sends those files to an interpreter, whose file extension is .php, all other files such as .html, .htm and others are not sent to PHP interpreter even if they contain PHP codes inside them.
- Once the file is sent to the PHP interpreter, it starts finding all of the opening and closing PHP tags and then process the PHP code inside these tags.
- PHP interpreter also checks that whether there is a database connection or not, if it finds database connection then it will send or retrieve data from the database.
- PHP scripts are interpreted on the web server, and outcome (HTML) is sent back to the client machine.

PHP Syntaxes

PHP is designed to work with HTML, so you can easily write and embed PHP code with HTML. A PHP code block begins with <?php tag and ends with ?> tag.

```
<?php
   //your php code goes here
?>
```

PHP with HTML

echo is a command in PHP for writing output data to the browser, and at the end of each PHP statement, we are required to write a (semicolon) as you see in the below example code. Otherwise, PHP will report a syntax error if you write another statement.

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Hello World Program in PHP</title>
</head>
<body>
   <?php echo "Hello World"; ?>
</body>
</html>
```

To run this PHP program, write the above code in any text editor and save it to your web server under the www directory with a .php extension.

Once it's done, start the server, go to localhost, and type in the path of your file, i.e., **http://localhost/HelloWorld.php**

PHP Tags

PHP is designed to work with HTML, so you can easily write and embed PHP code with HTML. A PHP code block begins with <?php tag and ends with ?> tag.

```
<?php
   //your php code goes here
?>
```

PHP with HTML

echo is a command in PHP for writing output data to the browser, and at the end of each PHP statement, we are required to write a (semicolon) as you see in the below example code. Otherwise, PHP will report a syntax error if you write another statement.

```
<!DOCTYPE html>

<html lang="en">

<head>

   <meta charset="UTF-8">

   <meta name="viewport" content="width=device-width, initial-scale=1.0">

   <title>Hello World Program in PHP</title>

</head>
```

```
<body>

    <?php echo "Hello World"; ?>

</body>

</html>
```

To run this PHP program, write the above code in any text editor and save it to your web server under the www directory with a .php extension.

Once it's done, start the server, go to localhost, and type in the path of your file,
i.e., **http://localhost/HelloWorld.php**

PHP Comments : The PHP interpreter ignores the comment block; it can be used anywhere in the program to add info about the program or code block, which can be helpful for the programmer to understand the code easily in the feature.

In PHP, we can use // or # to make a single-line comment and /* and */ to make a large comment block.

DATA Types

- Numeric
- String
- Boolean
- Array
- object
- Resource
- NULL

Numeric Data Types: There are two different numeric data types, which are used for number:

- Integer
- Double (floating point numbers)

Integer Data Type: Integer data type has full number values.

An integer data type is a non-decimal number between -2,147,483,648 and 2,147,483,647.

Rules for integers:

- An integer must have at least one digit
- An integer must not have a decimal point
- An integer can be either positive or negative
- Integers can be specified in: decimal (base 10), hexadecimal (base 16), octal (base 8), or binary (base 2) notation

```
<?php

$intValue = 100;

?>
```

Double Data Types: Floating point numbers are the contains Double.

```
<?php

$doubValue = 55.5;

?>
```

String Data Types: String data type contains letters or textual information. Value is assigned to the double or single quotes.

Converting between data types:

```
<?php

    $a = 9.88;

    echo (int) $a; // Outputs 9
```

?>

Values cannot be converted to certain types; for example, you cannot convert any value into a resource. However, you can convert a resource into a numerical or string data type, in which case PHP will return the numeric ID of the resource, or the string Resource id # followed by the resource ID.

PHP Boolean

A Boolean represents two possible states: TRUE or FALSE.

```
$x = true;

var_dump($x);
```

PHP Array:

An array stores multiple values in one single variable.

In the following example $cars is an array. The PHP var_dump() function returns the data type and value:

```
$cars = array("Volvo","BMW","Toyota");

var_dump($cars);
```

PHP Object:

Classes and objects are the two main aspects of object-oriented programming.

A class is a template for objects, and an object is an instance of a class.

When the individual objects are created, they inherit all the properties and behaviors from the class, but each object will have different values for the properties.

Let's assume we have a class named Car that can have properties like model, color, etc. We can define variables like $model, $color, and so on, to hold the values of these properties. When the individual objects (Volvo, BMW, Toyota, etc.) are created, they inherit all the properties and behaviors from the

class, but each object will have different values for the properties. If you create a __construct() function, PHP will automatically call this function when you create an object from a class.

```
class Car {
  public $color;
  public $model;
  public function __construct($color, $model) {
    $this->color = $color;
    $this->model = $model;
  }
  public function message() {
    return "My car is a " . $this->color . " " . $this->model . "!";
  }
}
$myCar = new Car("red", "Volvo");
var_dump($myCar);
```

PHP NULL Value

Null is a special data type which can have only one value: NULL.

A variable of data type NULL is a variable that has no value assigned to it.

If a variable is created without a value, it is automatically assigned a value of NULL.

Variables can also be emptied by setting the value to NULL:

```
$x = "Hello world!";

$x = null;

var_dump($x);
```

PHP echo and print Statements

echo and print are more or less the same. They are both used to output data to the screen.

The differences are small: echo has no return value while print has a return value of 1 so it can be used in expressions. echo can take multiple parameters (although such usage is rare) while print can take one argument. echo is marginally faster than print.

The echo statement can be used with or without parentheses: echo or echo().

```
echo "Hello";

//same as:

echo("Hello");
```

The print statement can be used with or without parentheses: print or print().

```
print "Hello";

//same as:

print("Hello");
```

Difference Between Print and ECHO

	"echo"	"print"
type	This is a type of output string.	This is a type of output string.
parenthesis	Not written with parenthesis.	It can or can not be written with parenthesis.
strings	It can output one or more strings.	It can output only one string at a time, and returns 1.
Functionality	echo is faster than print.	print is slower than echo.

var_dump

var_dump Using this function, we can display the value of a variable along with data type.

<?php
$x=100;
$y=”scott”;
var_dump($x);
var_dump($y);
?>
Output: int 100 string(5) scott

printf() : Using this function, we can print variables with the help of
format specifiers.
Example:
<?php
?>
$x=100;
$y=”scott”;

printf(“%d”,$x);

printf(“%s”,$y);

Output:

100 scott

print_r(): Using this function, we can display all elements of array and properties of object.

Example:

```
<?php
?>
$x=array(10,20,30);
print_r($x);
```

Output:

```
Array
(
    [0]→10
    [1]→20
    [2]→30
)
```

VARIABLES:

- Variable is an identifier which holds data or another one variable and whose value can be changed at the execution time of script.
- Syntax:
  $variablename=value;
- A variable starts with the $ sign, followed by the name of the variable
- A variable name must start with a letter or the underscore character
- A variable name can't start with a number.
- A variable name can only contain alpha-numeric characters and underscores(A-z, 0-9 and _)
- Variable names are case-sensitive ($str and $STR both are two different)

Example:

```
<?php
$str="Hello world!";
$a=5;
```

```
$b=10.5;
echo "String is: $str <br/>";
echo "Integer is: $x <br/>";
echo "Float is: $y <br/>";
?>
```

VARIABLE SCOPES

- Scope of a variable is defined as its extent in program within which it can be accessed, i.e. the scope of a variable is the portion of the program with in which it is visible or can be accessed.
- Depending on the scopes, PHP has three variable scopes:

  1. Local variables: The variables declared within a function are called local variables to that function and has its scope only in that particular function. In simple words it cannot be accessed outside that function. Any declaration of a variable outside the function with same name as that of the one within the function is a complete different variable.

  Example:

```
<?php
$num = 60;
function local_var()
{
}
$num = 50;
echo "local num = $num \n";
local_var();
echo "Variable num outside local_var() is $num \n";
?>
```

  Output: local num = 50
  Variable num outside local_var() is 60

  2. Global variables: The variables declared outside a function are called global variables. These variables can be accessed directly outside a function. To get access within a function we need to use the “global” keyword before the variable to refer to the global variable.

  Example:

```
<?php
```

```
$num = 20;
function global_var()
{
        global $num;
      echo "Variable num inside function : $num \n";
}
 global_var();
 echo "Variable num outside function : $num \n";
?>
```

Output: Variable num inside function : 20

Variable num outside function : 20

3. Static variable: It is the characteristic of PHP to delete the variable, ones it completes its execution and the memory is freed. But sometimes we need to store the variables even after the completion of function execution. To do this we use static keyword and the variables are then called as static variables.

Example:

```
<?php
function static_var()
{
    static $num = 5;
    $sum = 2;
        $sum++;
    $num++;
        echo $num "\n";
    echo $sum "\n";
}
static_var();
static_var();
?>
```

Output: 6 3

7

Example:

```
<?php
3
function myTest()
{
}
static $x = 0;
echo $x;
$x++;
myTest();
myTest();
myTest();
?>
```

Output: 0 1 2

CONSTANTS:

- Constants are name or identifier that can't be changed during the execution of the script. In php constants are define in two ways;
    - Using define() function
    - Using const keyword
- In php declare constants follow same rule variable declaration.
- Constant start with letter or underscore only.
- Create a PHP Constant :Create constant in php by using define() function.

  Syntax:

  define((name, value, case-insensitive)

  Name: Specifies the name of the constant

  Value: Specifies the value of the constant

  Case-insensitive: Specifies whether the constant name should be case-insensitive. Default is false

  Example:

```
<?php
define("MSG","Hello world!");
echo MSG;
?>
```

  Output: Hello world!

Example:

```
<?php
define("MSG","Hello world!");
//using case-sensitive
//using case-insensitive
echo msg;
?>
```

Output: Hello world!

Define constant using cons keyword in PHP

The const keyword defines constants at compile time. It is a language construct not a function. It is bit faster than define(). It is always case sensitive.

Example:

```
<?php
const MSG="Hello world!";
echo MSG;
?>
```

PHP Operators: Operators are used to perform operations on variables and values. PHP divides the operators in the following groups:

- Arithmetic operators
- Assignment operators
- Comparison operators
- Increment/Decrement operators
- Logical operators
- String operators
- Array operators

PHP Arithmetic Operators: The arithmetic operators are used to perform common arithmetical operations, such as addition, subtraction, multiplication etc. Here's a complete list of PHP's arithmetic operators:

Operator	Description	Example	Result
+	Addition	$x + $y	Sum of $x and $y
-	Subtraction	$x - $y	Difference of $x and $y.
*	Multiplication	$x * $y	Product of $x and $y.
/	Division	$x / $y	Quotient of $x and $y
%	Modulus	$x % $y	Remainder of $x divided by $y

PHP Assignment Operators: The assignment operators are used toassign values to variables.

Operator	Description	Example	Is The Same As
=	Assign	$x = $y	$x = $y
+=	Add and assign	$x += $y	$x = $x + $y
-=	Subtract and assign	$x -= $y	$x = $x - $y
*=	Multiply and assign	$x *= $y	$x = $x * $y
/=	Divide and assign quotient	$x /= $y	$x = $x / $y
%=	Divide and assign modulus	$x %= $y	$x = $x % $y

PHP Comparison Operators: The comparison operators are used tocompare two values in a Boolean fashion.

Operator	Name	Example	Result
==	Equal	$x == $y	True if $x is equal to $y
===	Identical	$x === $y	True if $x is equal to $y, and they are of the same type
!=	Not equal	$x != $y	True if $x is not equal to $y
<>	Not equal	$x <> $y	True if $x is not equal to $y

!==	Not identical	$x !== $y	True if $x is not equal to $y, or they are not of the same type
<	Less than	$x < $y	True if $x is less than $y
>	Greater than	$x > $y	True if $x is greater than $y

PHP Incrementing and Decrementing Operators

The increment/decrement operators are use t o increment/decrementa variable's value.

PHP Logical Operators

The logical operators are typically used to combine conditionalstatements.

Operator	Name
and	And
or	Or
xor	Xor
&&	And
||	Or
!	**Not**

PHP String Operators

There are two operators which are specifically designed for strings.

Operator	Name	Example	Result
+	Union	$x + $y	Union of $x and $y
==	Equality	$x == $y	True if $x and $y have the same key/value pairs

===	Identity	$x === $y	True if $x and $y have the same key/value pairs in the same order and of the same types
!=	Inequality	$x != $y	True if $x is not equal to $y
<>	Inequality	$x <> $y	True if $x is not equal to $y
!==	Non-identity	$x !== $y	True if $x is not identical to $y

DECISION MAKING

- PHP allows us to perform actions based on some type of conditionsthat may be logical or comparative.
- Based on the result of these conditions i.e., either TRUE or FALSE,an action would be performed.
- PHP provides us with four conditional statements:

1. if statement
2. if…else statement
3. if…elseif…else statement
4. switch statement

1. **if Statement:** The *if* statement is used to execute a block of code only if the specified condition evaluates to true. This is the simplest PHP's conditional statements and can be written like:

Syntax :

```
if (condition)
        {



        }
```

2. **if…else Statement:** You can enhance the decision making process by providing an alternative choice through adding an *else*

statement to the *if* statement. The *if...else* statement allows you to execute one block of code if the specified condition is evaluates to true and another block of code if it is evaluates to false. It can be written, like this:

Example:

```
<?php
    $x = -12;
    if ($x > 0)
        echo "The number is positive";
      else
        echo "The number is negative";
?>
```

Output: The number is negative

3. **if…elseif…else Statement:** he *if...elseif...else* a special statement that is used to combine multiple *if...else* statements.. We use this when there are multiple conditions of TRUE cases.

Example:

```
<?php
        $x = "August";
        if ($x == "January")
        {
                echo "Happy Republic Day";
        }
        elseif ($x == "August")
        {
                echo "Happy Independence Day!!!";
        }
        else
        {
                echo "Nothing to show";

        }
?>
```

Output: Happy Independence Day!!!

4. **switch Statement:** The "switch" performs in various cases i.e., it has various cases to which it matches the condition and appropriately executes a particular case block. It first evaluates an expression and then compares with the values of each case. If a case matches then the same case is executed. To use switch, we need to get familiar with two different keywords namely, break and default.

   The break statement is used to stop the automatic control flow intothe next cases and exit from the switch case.

   The default statement contains the code that would execute if noneof the cases match.

Syntax:

```
switch(expression)
{
    case value1:
        code to be executed if n==statement1;break;
    case value 2:
        code to be executed if n==statement2;break;
    case value 3:
        code to be executed if n==statement3;break;
    case value 4:
        code to be executed if n==statement4;break;
        ……
     default:
        code to be executed if n != any case;
}
```

LOOPS

- Loops are used to execute the same block of code again and again, until a certain

condition is met. The basic idea behind a loop is to automate the repetitive tasks within a program to save the time and effort. PHP supports four different types of loops.

1. for loop
2. while loop
3. do-while loop
4. foreach loop

for loop: This type of loops is used when the user knows in advance, how many times the block needs to execute. These type of loops are also known as entry-controlled loops. There are three main parameters to the code, namely the initialization, the testcondition and the counter.

Syntax:

```
for (initialization expression; test condition; update expression)
{
        // code to be executed
}
```

In for loop, a loop variable is used to control the loop. Firstinitialize this loop variable to some value, then check whether this variable is less than or greater than counter value. If statement is true, then loop body is executed and loop variable gets updated. Steps are repeated till exit condition comes.

1. **while loop:** The while loop is also an entry control loop like for loops i.e., it first checks the condition at the start of the loop and if its true then it enters the loop and executes the block of statements, and goes on executing it as long as the condition holds true.

Syntax:

```
while (if the condition is true)
{
```

```
        // code is executed
    }
```

Example:

```
<?php
    $num = 2;
     while ($num < 12)
    {
        $num += 2;
        echo $num, "\n";
    }


?>
```

Output: 4 6 8 10 12

1. **do-while loop:** This is an exit control loop which means that it firstenters the loop, executes the statements, and then checks the condition. Therefore, a statement is executed at least once on using the do…while loop. After executing once, the program is executedas long as the condition holds true.

Syntax:

```
    do
    {
            //code is executed
} while (if condition is true);
```

1. **foreach loop:** The foreach statement is used to loop through arrays. For each pass

the value of the current array element isassigned to $value and the array pointer is moved by one and in the next pass next element will be processed.

Syntax:

```
foreach (array_element as value)
{
        //code to be executed
}
```

Break:

The PHP break keyword is used to terminate the execution of a loop prematurely. The break statement is placed inside the statement block. It gives you full control and whenever you want to exit from the loop you can come out. After coming out of a loop immediate statement to the loop will be executed.

CREATE AN ARRAY IN PHP

In PHP, the array() function is used to create an array.

Syntax: array( );

TYPES OF ARRAY IN PHP

There are three types of array in PHP, which are given below.

1. Indexed arrays - Arrays with a numeric index
2. Associative arrays - Arrays with named keys
3. Multidimensional arrays - Arrays containing one or more arrays

1. Indexed Arrays

The index can be assigned automatically (index always starts at 0).

Example

```
<?php
    $student = array("Harry", "Varsha", "Gaurav");
    echo "Class 10th Students " . $student[0] . ", " . $student[1] . "and " .
$student[2] . ".";
?>
```

Arrays using for Loop Example

```
<?php
    $student = array("Harry", "Varsha", "Gaurav");
    $arrlength = count($student); for($i =
    0; $i < $arrlength; $i++)
    {
        echo $student[$i];echo
        "<br>";
    }
?>
```

1. Associative Arrays in PHP

In this type of array; arrays use named keys that you assign to them.

Syntax

```
$age = array("Harry"=>"10", "Varsha"=>"20", "Gaurav"=>"30"); or

$age['Harry'] = "10";

$age['Varsha'] = "20";

$age['Gaurav'] = "30";
```

2. Multidimensional Arrays in PHP

A multidimensional array is an array containing one or more arrays. For a two-dimensional array you need two indices to select an element **Example**

```
<?php

$student = array( array("Harry",300,11), array("Varsha",400,10),

array("Gaurav",200,8), array("Hitesh",220,8));



echo $student[0][0].": Marks: ".$student[0][1].", Class: ".$student[0][2].".<br>";

echo $student[1][0].": Marks: ".$student[1][1].", Class: ".$student[1][2].".<br>";

echo $student[2][0].": Marks: ".$student[2][1].", Class: ".$student[2][2].".<br>";

echo $student[3][0].": Marks: ".$student[3][1].", Class: ".$student[3][2].".<br>";

?>
```

Output: Harry: Marks: 300 Class: 11

Varsha: Marks: 400 Class: 10

Gaurav: Marks: 200 Class: 8

Hitesh: Marks: 220 Class: 8

SORT FUNCTIONS FOR ARRAYS

ARRAY FUNCTIONS

Function	Description
array()	Creates an array
array_combine()	Creates an array by using the elements from one "keys" array and one "values" array
array_count_values()	Counts all the values of an array
array_diff()	Compare arrays, and returns the differences (compare values only)
array_diff_assoc()	Compare arrays, and returns the differences (compare keys and values)
array_diff_key()	Compare arrays, and returns the differences (compare keys only)
array_fill()	Fills an array with values
array_fill_keys()	Fills an array with values, specifying keys
array_filter()	Filters the values of an array using a callback function
array_flip()	Flips/Exchanges all keys with their associated values in an array
array_intersect()	Compare arrays, and returns the matches (compare values only)

array_keys()	Returns all the keys of an array
array_map()	Sends each value of an array to a user-made function, which returns new values
array_merge()	Merges one or more arrays into one array
array_merge_recursive()	Merges one or more arrays into one array recursively
array_multisort()	Sorts multiple or multi-dimensional arrays
array_pad()	Inserts a specified number of items, with a specified value, to an array
array_pop()	Deletes the last element of an array
array_product()	Calculates the product of the values in an array
array_push()	Inserts one or more elements to the end of an array
array_rand()	Returns one or more random keys from an array
array_reduce()	Returns an array as a string, using a user-defined function
array_replace()	Replaces the values of the first array with the values from following arrays
array_replace_recursive()	Replaces the values of the first array with the values from following arrays recursively
array_reverse()	Returns an array in the reverse order

FUNCTIONS

- A function is a block of code written in a program to perform some specific task. Functions take information's as parameter, execute a block of statements or perform operations on these parameters and return the result.

- PHP provides us with two major types of functions:

1. **Built-in functions:** PHP provides us with huge collection of built-in library functions. These functions are already coded and stored in form of functions. To use those we just need to call them as per our requirement like, var_dump, fopen(), print_r(), gettype() and so on.

2. **User Defined Functions:** Apart from the built-in functions, PHP allows us to create our own customized functions called the user- defined functions.

- Why should we use functions?

3. Reusability: If we have a common code that we would like to use at various parts of a program, we can simply contain it within a function and call it whenever required. This reduces the time and effort of repetition of a single code. This can be done both within a program and also by importing the PHP file, containing the function, in some other program

4. Easier error detection: Since, our code is divided into functions, we can easily detect in which function, and the error could lie and fix them fast and easily.

5. Easily maintained: If anything or any line of code needs to be changed, we can easily change it inside the function and the change will be reflected everywhere, where the function is called. Hence, easy to maintain.

CREATING A FUNCTION

- While creating a user defined function we need to keep few things in mind:

1. Any name ending with an open and closed parenthesis is a function.

2. A function name always begins with the keyword function.

3. To call a function we just need to write its name followed by parenthesis.

4. A function name cannot start with a number. It can start with an alphabet or underscore.

5. A function name is not case-sensitive.

Syntax:

```
function function_name()
{
    Executable code;
}
```

FUNCTION PARAMETERS OR ARGUMENTS

- The information or variable, within the function's parenthesis, are called parameters.
- These are used to hold the values executable during runtime.
- A user is free to take in as many parameters as he wants, separated with a comma(,) operator. These parameters are used to accept inputs during runtime.
- While passing the values like during a function call, they are called arguments.
- An argument is a value passed to a function and a parameter is used to hold those arguments. In common term, both parameterand argument mean the same.

Syntax:

```
function function_name($first_parameter, $second_parameter)
{
    executable code;
}
```

SETTING DEFAULT VALUES FOR FUNCTION PARAMETER

- PHP allows us to set default argument values for function parameters.
- If we do not pass any argument for a parameter then PHP will usethe default set value for this parameter in the function call.

RETURNING VALUES FROM FUNCTIONS

- Functions can also return values to the part of program from whereit is called.
- The return keyword is used to return value back to the part ofprogram,

from where it was called.

- The returning value may be of any type including the arrays andobjects.
- The return statement also marks the end of the function and stopsthe execution after that and returns the value.

Example:

```
<?php
    function pro($num1, $num2, $num3)
    {
        $product = $num1 * $num2 * $num3;return
        $product;
    }
    $retValue = pro(2, 3, 5);
    echo "The product is $retValue";
?>
```

Output: The product is 30

PARAMETER PASSING TO FUNCTIONS

- PHP allows us two ways in which an argument can be passed into a function:

1. **Pass by Value:** On passing arguments using pass by value, the value of the argument gets changed within a function, but the original value outside the function remains unchanged. That means a duplicate of the original value is passed as an argument.
2. **Pass by Reference:** On passing arguments as pass by reference, the original value is passed. Therefore, the original value gets altered. In pass by reference we actually pass the address of the value, where it is stored using ampersand sign(&).

Example:

```
<?php
    function val($num)
```

```
{
        $num += 2; return
        $num;
}
function ref (&$num)
{
    $num += 10;
    return $num;
}
$n = 10;
val($n);
echo "The original value is still $n \n";ref($n);
echo "The original value changes to $n";
?>
```

Output: The original value is still 10

The original value changes to 20

VARIABLE FUNCTIONS

- PHP supports the concept of variable functions. This means that ifa variable name has parentheses appended to it, PHP will look for a function with the same name as whatever the variable evaluates to,and will attempt to execute it.
- Among other things, this can be used to implement callbacks,function tables, and so forth.

Example:

```
function foo()
```

```
{
    echo "hi<br />";
}

function bar($arg = '')
{
    echo " argument was '$arg'.<br />n";
}
function echoit($string)
{
    echo $string;
}

$func = 'foo';
$func();        // This calls foo()
$func = 'bar';
$func('test'); // This calls bar()
$func = 'echoit';
$func('test'); // This calls echoit()
?>
```

Output: hi

argument was 'test'.test

RECURSIVE FUNCTIONS

- A recursive function is a function that calls itself again and again until a condition is satisfied.
- Recursive functions are often used to solve complex mathematical calculations, or to process deeply nested structures e.g., printing all the elements of a deeply nested array.

Example:

```
<?php
        function factorial($n)
        {
                if ($n==0)
                {


                }
                else
                {


                }
        }
return 1;
return $n *
factorial($n - 1);
```

```
    echo
    factorial(
    4), "\n";
    echo
    factorial(
    10), "\n";
?>
```

Output: 24

3628800

STRINGS: String is a sequence of characters.For example ‘rgmcet’ or ”rgmcet” is string.Everything inside the quotes single(‘ ‘) or double(“ “) in php can be treated as string. There are 2 ways to specify string in PHP

1. **single quoted strings:** We can create a string by enclosing text in a singlequote. This type of strings does not process special characters inside quotes.

Example

```
<?php
    $str='college';
    echo ‘ welcome to $str ’;
?>
```

Output: welcome to $str

In the above program the echo statement prints the variable name rather than printing the contents of the variables.this is because single quote strings in php does not process special characters.Hence the string is unable to identify the $ sign as start of a variable name.

2. **double quoted strings:** we can specify string by enclosing text within double quotes. This type of strings can process special characters inside quotes.

Example

```
<?php
        echo “Hello php I am double quote String\n";
        $str="college";
        echo “ welcome to $str”;
?>
```

Output: Hello php I am double quote String

welcome to college

In the above program we can see that the double quote string is processing the special characters according to their properties.The \n character is not printed and is considered as a newline .Also instead of the variable name college is printed

CONCATENATION OF TWO STRINGS

There are two string operators. The first is the concatenation operator (‘.‘), which returns the concatenation of its right and left arguments. The second is the concatenating assignment operator (‘.=‘), which appends the argument on the right side to the argument on the left side.

Example1:

```
<?php
        $a = 'Hello';
        $b = 'World!';
        $c = $a.$b;
        echo " $c \n";
?>
```

Output: HelloWorld!

STRING FUNCTIONS IN PHP

PHP have lots of predefined function which is used to perform operation withstring some functions are:

strlen(): strlen() function returns the length of a string.

Example

```
<?php
        echo strlen("Hello world!");
?>
```

Output: *12*

str_word_count(): str_word_count() function is used to count numbers ofwords in given string.

Example

```
<?php
        echo str_word_count("Hello world!");
?>
```

Output: *2*

strrev(): strrev() function is used to revers any string.

Example

```
<?php
        echo strrev("Hello world!");
?>
```

Output: !dlrow olleH

strtolower(): strtolower() function is used to convert uppercase latter intolowercase latter.

Example

```
<?php
        $str="Hello friends i am HITESH";
```

```
    $str=st
    rtolowe
    r($str);
    echo
    $str;
?>
```

Output: hello friends i am hitesh

strtoupper(): PHP strtoupper() function is used to convert lowercase latter intouppercase latter.

ucwords(): ucwords() function is used to convert first letter of every word intoupper case.

Output: Hello Friends I Am Hitesh

ucfirst(): ucfirst() function returns string converting first character intouppercase. It doesn't change the case of other characters.

lcfirst(): lcfirst() function is used to convert first character into lowercase. Itdoesn't change the case of other characters.

Example

```
<?php
    function firstLower($string)
    {
        return(lcfirst($string));
    }
    $string="WELCOME to
    GeeksforGeeks";echo
    (firstLower($string));
?>
```

Output: wELCOME to GeeksforGeeks

strstr()(or) **strchr():** The strchr() function searches for the first

occurrence of astring inside another string.

Syntax: strchr(string,search)

Example:

```
<?php
echo strchr("Hello world!","world");
?>
```

output: world!

Example: Search a string for the ASCII value of "o" and return the rest of thestring:

```
<?php
echo strchr("Hello world!",111);
?>
```

Output: o world!

Example:

```
<?php
$originalStr = "geeks for geeks";
$searchStr = "geeks" ;
echo strchr($originalStr, $searchStr);
?>
```

Output: geeks for geeks

Program 2: Program to demonstrate strchr() fucntion when word is not found.

```
<?php
$originalStr = "geeks for geeks";
$searchStr = "gfg" ;
echo strchr($originalStr, $searchStr);
?>
```

Output: -NIL-

strcmp(): Compare two strings with case-sensitive manner

Syntax: strcmp(string1,string2)

This function returns:

0 - if the two strings are equal

<0 - if string1 is less than string2

>0 - if string1 is greater than string2

Example:

```
<?php
echo
strcmp("He
llo","Hello
");echo
"<br>";
echo strcmp("Hello","hELLo");
?>
```

Output: 0

-1

Example:

```
<?php
echo strcmp("Hello world!","Hello world!")."<br>";
echo strcmp("Hello world!","Hello")."<br>"; // string1 is  greater
than  string2 echo  strcmp("Hello  world!","Hello  world!
Hello!")."<br>"; // string1 is less thanstring2
?>
```

Output: *0*

7

-7

strcasecmp(): Compare two strings with case-insensitive manner

Syntax: strcasecmp(string1,string2)

This function returns:

0 - if the two strings are equal

<0 - if string1 is less than string2

>0 - if string1 is greater than string2

Example:

```
<?php
echo
strcasecmp("Hello",
"HELLO");echo
strcasecmp("Hello",
"hELLo");
?>
```

Output: 0

0

Example:

```
<?php
echo strcasecmp("Hello world!","HELLO WORLD!"); // The two
strings are equalecho strcasecmp("Hello world!","HELLO"); //
String1 is greater than string2 echo strcasecmp("Hello
world!","HELLO WORLD! HELLO!"); // String1 is lessthan
string2
?>
```

Output: *0*

7

-7

strncmp(): The strncmp() is used to compare first n character of two strings. This function is case-sensitive which points that capital and small cases will be treated differently, during comparison.

Syntax: strncmp($str1, $str2, $len) This function returns:

0 - if the two strings are equal

<0 - if string1 is less than string2

>0 - if string1 is greater than string2

Example:

```
<?php
$str1 = "Geeks for Geeks ";
$str2 = "Geeks for Geeks ";
// Both the strings are equal
$test=strncmp($str1, $str2, 16 );
echo "$test";
?>
```

Output: *0*

Example:

```
?php
// Input strings
$str1 = "Geeks for Geeks ";
$str2 = "Geeks for ";
```

```
$test=strncmp($str1, $str2, 16 );
// In this case the second
string is smallerecho
"$test";
?>
```

Output: 6

Example:

```
<?php
// Input Strings
$str1 = "Geeks for ";
$str2 = "Geeks for Geeks ";
$test=strncmp($str1, $str2, 16 );
// In this case the first
string is smallerecho
"$test";
?>
```

Output: -6

strncasecmp(): The strncmp() is used to compare first n character of two strings. This function is case-sensitive which points that capital and small cases will be treated differently, during comparison.

Syntax: strncasecmp($str1, $str2, $len) This function returns:

0 - if the two strings are equal

<0 - if string1 is less than string2

>0 - if string1 is greater than string2

strpos() It enables searching particular text within a string. It works simply by matching the specific text in a string. If found,

then it returns the specific position. If not found at all, then it will return “False”.

Syntax: Strpos(string,text);

Example

```
<?php
echo strpos(“Welcome to Cloudways”,”Cloudways”);
?>
```

Output: 11

Str_replace(): It used for replacing specific text within a string.

Syntax: Str_replace(string to be replaced,text,string)

Example

```
<?php
echo str_replace(“cloudways”, “the programming world”, “Welcome to cloudways”);
?>
```

Output: Welcome to the programming world

str_repeat(): This function is used for repeating a string a specific number oftimes.

Syntax: Str_repeat(string,repeat)

Example

```
<?php
echo str_repeat(“=”,13);
?>
```

Output: ==========

substr(): This function you can display or extract a string from a particularposition.

Syntax: ***substr(string,start,length)***

Example

```
<?php
echo substr(“Welcome to
Cloudways”,6).”<br>”; echo
substr(“Welcome to
Cloudways”,0,10).”<br>”;
?>
```

Output: e to Cloudways

Welcome to

chunk_split(): The chunk_split() function splits a string into a series of smallerparts.

Syntax: chunk_split(string,length,end)

Example: Split the string after each sixth character and add a "..." after eachsplit:

```
<?php
$str = "Hello world!";
echo chunk_split($str,6,"...");
?>
```

Output: Hello ...world!...

trim():Removes whitespace or other characters from both sides of a stringParameter Description

string	Required. Specifies the string to check
charlist	Optional. Specifies which characters to remove from the string. If omitted, all of the following characters are removed

"\0" - NULL

"\t" - tab

"\n" - new line

"\x0B" - vertical tab

"\r" - carriage return

" " - ordinary white space

Example:

```
<?php
echo trim(" testing ").”<br>”;
echo trim(" testing ", " teng");

?>
```

Output: testing

sti

Example:

```
<?php
echo trim("
testing
").”<br>”;
echo trim("
testing ", "
teng");
?>
```

ltrim(): Removes whitespace or other characters from the left side of a string

Syntax: ltrim(string,charlist)

Output: Hello World!

World!

rtrim(): Removes whitespace or other characters from the right side of a string

Syntax: rtrim(string,charlist)

Output: Hello World!

Hello

Function	*Description*
chop()	Removes whitespace or other characters from the rightend of a string
chr()	Returns a character from a specified ASCII value count_chars() Returns information about characters used in a stringecho() Outputs one or more strings
explode()	Breaks a string into an array
fprintf()	Writes a formatted string to a specified output stream
parse_str()	Parses a query string into variables
print()	Outputs one or more strings
printf()	Outputs a formatted string
sprintf()	Writes a formatted string to a variable
sscanf()	Parses input from a string according to a format
str_getcsv()	Parses a CSV string into an array
str_ireplace()	Replaces some characters in a string (case-insensitive)str_pad() Pads a string to a new length
str_replace()	Replaces some characters in a string (case-sensitive)str_shuffle() Randomly shuffles all characters in a string str_split() Splits a string into an array
strcspn()	Returns the number of characters found in a string before any part of some specified characters are found
stripos()	Returns the position of the first occurrence of a stringinside another string (case-insensitive)
stristr()	Finds the first occurrence of a string inside anotherstring (case-insensitive)
strrchr()	Finds the last occurrence of a

	string inside anotherstring
	strripos() Finds the position of the last occurrence of a stringinside another string (case-insensitive)
strrpos()	Finds the position of the last occurrence of a stringinside another string (case-sensitive)
strspn()	Returns the number of characters found in a string that contains only characters from a specified charlist
substr()	Returns a part of a string
substr_compare()	Compares two strings from a specified start position (binary safe and optionally case-sensitive)
substr_count()	Counts the number of times a substring occurs in a string
substr_replace()	Replaces a part of a string with another string